

call-socket.js — Line-by-Line Explainer

A walkthrough of the WebRTC audio-call signaling module, explaining what every part of the code does and why it's there.

Before diving in: this file does not transmit audio. Socket.IO here is only a **signaling channel** — a way for two clients to exchange the setup information (SDP offers/answers and ICE candidates) that WebRTC needs to open a direct peer-to-peer audio connection. Once that connection is open, the audio itself never touches this server.

1. File Header & State Setup

The comment block

```
/**
 * call-socket.js
 * WebRTC signaling layer for 1:1 audio calls...
 */
```

This is a JSDoc-style comment explaining the file's purpose up front. It's not executed — it's documentation for the next developer (or future you) so they don't need to reverse-engineer the intent from the code alone. It explicitly calls out the most important architectural fact: this file only handles signaling, not audio itself.

const activeCalls = new Map();

```
const activeCalls = new Map();
```

This is the in-memory registry of every call currently in progress. A JavaScript **Map** is used instead of a plain object because it gives clean `.get()`, `.set()`, `.has()`, and `.delete()` methods without the footguns of using arbitrary strings as object keys (like accidentally colliding with built-in properties such as `__proto__`).

Each entry is keyed by a unique **callId** and stores an object with the caller's id, the callee's id, the call's status (*ringing* or *active*), and a timestamp for when it started.

const userToCall = new Map();

```
const userToCall = new Map();
```

A second lookup table, this time mapping a **userId** directly to the **callId** they're currently part of. This exists purely for speed and convenience: it lets the code instantly answer "is this user already on a call?" or "which call should I clean up when this user disconnects?" without having to loop through every entry in `activeCalls` to find it.

2. Helper Functions

function makeCallId(userA, userB) { ... }

```
function makeCallId(userA, userB) {  
  return [userA, userB].sort().join(":") + ":" + Date.now();  
}
```

Generates a unique identifier for a new call between two users.

[userA, userB].sort() puts the two user IDs into a predictable, consistent order regardless of who called who (so a call between "userX" and "userY" always sorts the same way).

.join(":") combines them into a single string like userX:userY.

+ ":" + Date.now() appends the current timestamp in milliseconds. This guarantees uniqueness — without it, if the same two users called each other twice in a row, the second call's ID could collide with a leftover reference to the first one.

The result looks something like userX:userY:1735689600000, and this string doubles as a Socket.IO **room name** later in the file.

function endCallInternal(callId, reason, io) { ... }

```
function endCallInternal(callId, reason, io) {  
  const call = activeCalls.get(callId);  
  if (!call) return;  
  
  const { callerId, calleeId } = call;  
  
  io.to(callId).emit("call:ended", { callId, reason });  
  
  userToCall.delete(callerId);  
  userToCall.delete(calleeId);  
  activeCalls.delete(callId);  
}
```

A shared cleanup routine used by every code path that can end a call (hang up, reject, or disconnect). Centralizing this in one function avoids repeating the same three cleanup steps in multiple event handlers, which would risk them drifting out of sync over time.

activeCalls.get(callId) looks up the call; if it doesn't exist (already ended, or a bad/stale ID), the function exits immediately via **if (!call) return;** — this guards against double-cleanup or acting on garbage input.

io.to(callId).emit("call:ended", ...) broadcasts an event to everyone in that Socket.IO room (i.e. both participants) telling their clients the call is over and why (reason might be "ended", "rejected", or "disconnected").

The final three lines remove all trace of the call from both in-memory maps, freeing up the userId slots so those users can start or receive new calls.

3. registerCallHandlers(io, socket, userId, getSocketsByUserId)

This is the main exported function. It's meant to be called once per socket, right after your existing JWT authentication succeeds in `life-update.js`. It takes the already-verified `userId` as a parameter — it does not re-check identity itself, it trusts the caller.

3.1 — `call:invite` (starting a call)

```
socket.on("call:invite", ({ calleeId }) => {
  if (!calleeId) return;

  if (userToCall.has(userId)) {
    return socket.emit("call:error", { message: "You are already on a call." });
  }

  const calleeSockets = getSocketsByUserId(calleeId);
  if (calleeSockets.length === 0) {
    return socket.emit("call:error", { message: "User is offline." });
  }

  if (userToCall.has(calleeId)) {
    return socket.emit("call:busy", { calleeId });
  }

  const callId = makeCallId(userId, calleeId);

  activeCalls.set(callId, {
    callerId: userId,
    calleeId,
    status: "ringing",
    startedAt: Date.now(),
  });
  userToCall.set(userId, callId);
  userToCall.set(calleeId, callId);

  socket.join(callId);

  calleeSockets.forEach((sId) => {
    io.to(sId).emit("call:incoming", {
      callId,
      from: userId,
    });
  });

  socket.emit("call:ringing", { callId, calleeId });
});
```

This listens for the client-side event **`call:invite`**, which the caller's app emits when the user taps "Call". The payload is destructured to pull out just `calleeId` — the person being called.

`if (!calleeId) return;` guards against a malformed request with no target.

`userToCall.has(userId)` checks whether the caller is already mid-call. If so, it sends back a `call:error` event instead of letting them start a second, conflicting call.

getSocketsByUserId(calleeId) is the function from life-update.js we discussed earlier — it returns every active socket.id for that user. If it's an empty array, the callee has no live connection at all, so we respond with a call:error saying they're offline.

userToCall.has(calleeId) checks if the person being called is already on a different call. If so, we emit call:busy back to the caller — this is the classic "line is busy" signal.

Once all guards pass, **makeCallId(userId, calleeId)** generates the unique call/room identifier described above.

activeCalls.set(...) registers the new call with status "ringing", and **userToCall.set(...)** is called twice to mark both the caller and callee as busy with this call — this is what makes the busy-check above work for future invites.

socket.join(callId) adds the caller's own socket to a Socket.IO room named after the callId. Rooms are Socket.IO's built-in grouping mechanism: anything emitted "to" that room reaches everyone who has joined it.

The **calleeSockets.forEach(...)** loop sends a call:incoming event to every device the callee is logged in on (phone, laptop, tablet, etc.) so all of them ring simultaneously, not just one.

Finally, **socket.emit("call:ringing", ...)** tells the caller's own client "your call is going through" so their UI can show a ringing state.

3.2 — call:accept (callee picks up)

```
socket.on("call:accept", ({ callId }) => {
  const call = activeCalls.get(callId);
  if (!call || call.calleeId !== userId) return;

  call.status = "active";
  socket.join(callId);

  io.to(callId).emit("call:accepted", { callId });
});
```

Fired when the callee taps "Answer" on their device.

activeCalls.get(callId) retrieves the call record. The guard **if (!call || call.calleeId !== userId) return;** does two things at once: confirms the call still exists, and confirms the person accepting is actually the intended callee (not the caller trying to accept their own call, and not some unrelated user spoofing a callId).

call.status = "active" updates the call's state now that both sides are connected/ready.

socket.join(callId) puts the callee's socket into the same Socket.IO room the caller joined earlier, meaning from this point on both sockets can exchange messages via io.to(callId) or socket.to(callId).

io.to(callId).emit("call:accepted", ...) notifies everyone in the room (both caller and callee) that the call was picked up, so both UIs can move from "ringing" to "connected" and begin the WebRTC handshake.

3.3 — call:reject (decline or cancel)

```
socket.on("call:reject", ({ callId }) => {
  const call = activeCalls.get(callId);
```

```

    if (!call) return;
    if (call.callerId !== userId && call.calleeId !== userId) return;

    endCallInternal(callId, "rejected", io);
  });

```

Handles both "callee declines" and "caller cancels before it's picked up" — the same event covers both because the end result is identical: the call never connects.

if (!call) return; ignores requests for calls that don't exist (already ended, bad ID).

if (call.callerId !== userId && call.calleeId !== userId) return; is an authorization check: only the two people actually on this call are allowed to reject it. Without this, any authenticated user could guess or obtain a callId and kill someone else's call.

If both checks pass, it delegates to the shared **endCallInternal** helper with reason "rejected", which notifies both parties and cleans up the maps.

3.4 — call:offer / call:answer / call:ice-candidate (WebRTC relay)

```

socket.on("call:offer", ({ callId, sdp }) => {
  if (!activeCalls.has(callId)) return;
  socket.to(callId).emit("call:offer", { callId, sdp, from: userId });
});

socket.on("call:answer", ({ callId, sdp }) => {
  if (!activeCalls.has(callId)) return;
  socket.to(callId).emit("call:answer", { callId, sdp, from: userId });
});

socket.on("call:ice-candidate", ({ callId, candidate }) => {
  if (!activeCalls.has(callId)) return;
  socket.to(callId).emit("call:ice-candidate", { callId, candidate, from: userId });
});

```

These three handlers are almost identical, and together they form the actual WebRTC handshake relay. The server treats the payloads (sdp, candidate) as opaque data — it never reads or understands their contents, it just forwards them.

call:offer: after the callee accepts, the caller's browser generates an SDP (Session Description Protocol) "offer" describing its audio capabilities and connection info, and sends it here to be relayed.

call:answer: the callee's browser receives that offer, generates its own SDP "answer", and sends it back through this same relay mechanism.

call:ice-candidate: as each side discovers possible network paths (ICE candidates) for connecting directly, they trickle these to each other through this event, often many times per call as different network routes are discovered.

In every case, **socket.to(callId)** (note: socket.to, not io.to) broadcasts to everyone else in the room *except the sender* — you don't want to echo a client's own offer back to itself.

The **if (!activeCalls.has(callId)) return;** guard on each simply discards messages for calls that no longer exist, e.g. if one side hung up mid-handshake and a stale ICE candidate arrives late.

from: `userId` is attached to each relayed message so the receiving client knows who it came from, useful if you extend this to group calls later.

3.5 — `call:end` (hang up)

```
socket.on("call:end", ({ callId }) => {
  const call = activeCalls.get(callId);
  if (!call) return;
  if (call.callerId !== userId && call.calleeId !== userId) return;

  endCallInternal(callId, "ended", io);
});
```

Structurally identical to **`call:reject`**, but used once a call is already active rather than before it's picked up. The distinction between "rejected" and "ended" as reasons lets the client UI show different messages (e.g. "Call declined" vs "Call ended") even though the server logic is the same — both just call **`endCallInternal`**.

3.6 — `disconnect` cleanup

```
socket.on("disconnect", () => {
  const callId = userToCall.get(userId);
  if (callId) {
    endCallInternal(callId, "disconnected", io);
  }
});
```

A safety net for the case where a user doesn't hang up cleanly — their app crashes, they lose internet, or they close the browser tab mid-call. `Socket.IO` automatically fires its built-in `disconnect` event whenever a socket connection drops, for any reason.

`userToCall.get(userId)` checks whether this user was part of an active call at the moment they disconnected.

If so, **`endCallInternal`** is called with reason "disconnected", ensuring the other participant is notified and the call state doesn't get stuck open forever with a ghost participant.

4. Module Export

`module.exports = { registerCallHandlers };`

```
module.exports = { registerCallHandlers };
```

Exposes only **`registerCallHandlers`** to the rest of the app. Everything else in this file — `activeCalls`, `userToCall`, `makeCallId`, `endCallInternal` — stays private to this module. This is intentional encapsulation: no other file can directly poke at the call state or bypass the validation logic; the only way in is through the function that sets up the event listeners.

5. Known Limitations

A few things worth knowing before relying on this in production:

In-memory state. `activeCalls` and `userToCall` are plain JS Maps living in this process's memory. A server restart wipes all call state, and if you ever run multiple server instances behind a load balancer, each instance has its own separate Maps — two users landing on different instances won't be able to see each other as busy or reachable. Moving this into Redis fixes it.

No TURN server configured. This file only relays signaling messages; the actual peer-to-peer connection still needs STUN/TURN servers configured on the client's `RTCPeerConnection`. Without a TURN server, calls between users behind strict NATs or symmetric firewalls can fail to connect even though signaling succeeded.

1:1 only. The room-based design (two sockets per `callId`) assumes exactly two participants. Group calls would need either a mesh topology (each client connects to every other client) or a media server (SFU) to scale.

No authorization beyond caller/callee identity. The code checks that the acting user is part of the call, but doesn't check things like whether the caller and callee are allowed to contact each other at the application level (e.g. blocked users, contact lists) — that logic would need to live in `call:invite` before the call is created.